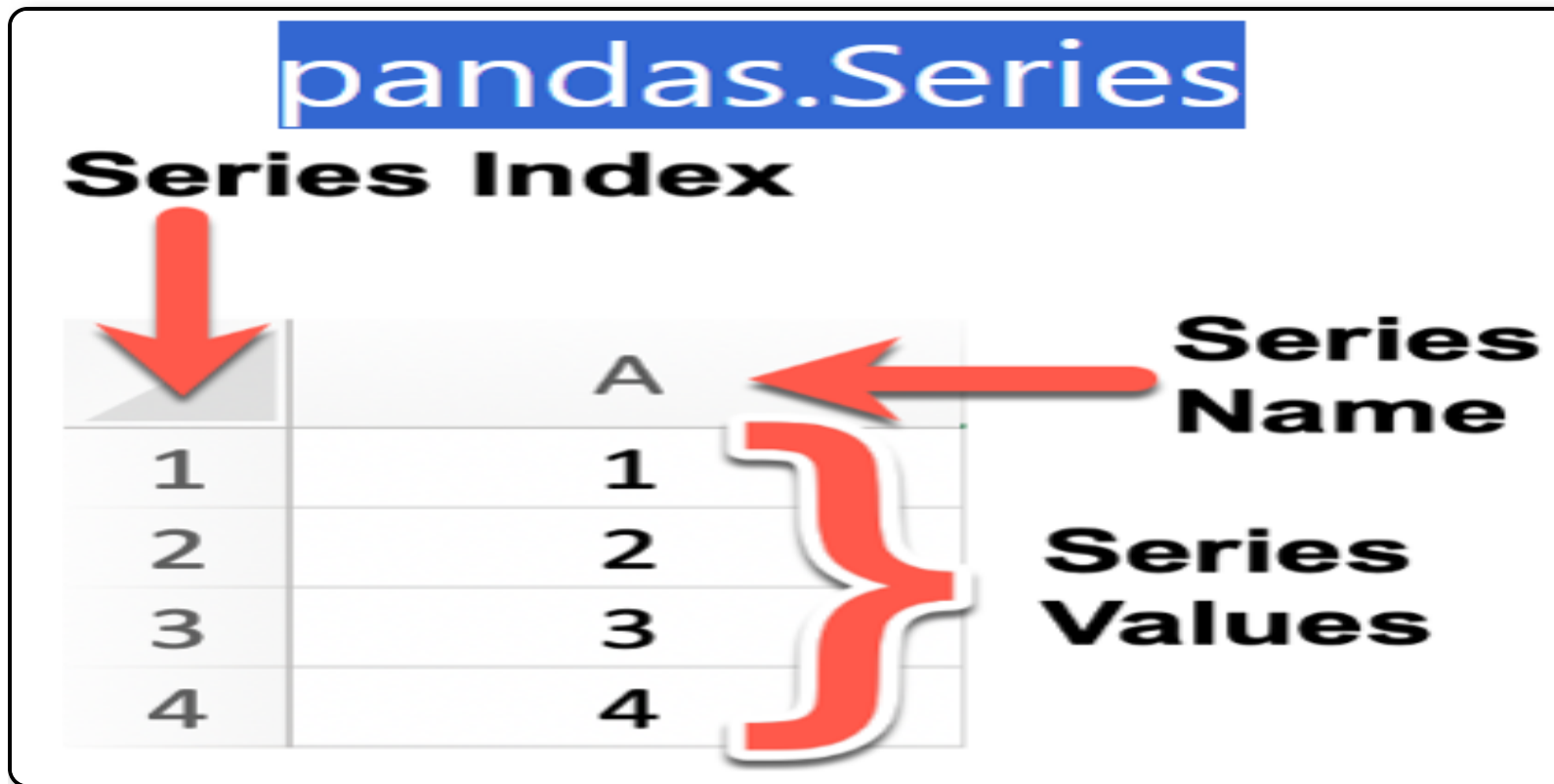


None



1st Edition

Why This E-book?

"The aim of this ebook is to give you the 'aha' moment right away at the start of learning a new concept."

- Practical step By Step Guide With Simple Examples
- Visual Illustrations and Interactive
- Simple Datasets
- Comprehensive Coverage (pandas Documentation used as reference)

#Import Libraries

```
import pandas as pd, numpy as np
```

Contents

[What Makes This Ebook Unique](#)

[Introduction](#)

["pandas.Series" Syntax](#)

[The "data" Parameter](#)

[The "index" Parameter](#)

[The "name" Parameter](#)

[The "copy" Parameter](#)

[Other ebooks](#)

[Author Biography](#)

[Contacts and Social Media](#)

[Sources & References](#)[Thank You!](#)

Introduction

Series is a One-dimensional ndarray with axis labels (including time series).

```
data = [10, 20, 30, 40, 50]
```

```
pd.Series(data)
```

series from data:

```
0    10
```

```
1    20
```

```
2    30
```

```
3    40
```

```
4    50
```

```
dtype: int64
```

pandas.Series syntax

```
class pandas.Series(data=None, index=None, dtype=None, name=None, copy=None, fastpath=_NoDefault.no_default)
```

Parameters

The `data` parameter

The `data` parameter accepts array-like, iterable, dict, or scalar value. It contains data stored in Series. If `data` is a dict, argument order is maintained.

When "`data`" is Array-like.

This includes lists, NumPy arrays, or anything that can be converted into a sequence of value

series from List:

```
List = [1, 2, 3, 4]
```

```
pd.Series(List)
```



```
0    1
1    2
2    3
3    4
dtype: int64
```

series from NumPy array:

```
Array = np.array([1, 2, 3, 4])
```

```
pd.Series(Array)
```



```
0    1
1    2
2    3
3    4
dtype: int64
```

When "`data`" is Iterable.

This includes any Python object that can be iterated over, such as lists, tuples, or custom iterators.

series from range:

```
pd.Series(range(1,6))
```



0	1
1	2
2	3
3	4
4	5

dtype: int64

series from tuple:

```
pd.Series((1, 2, 3, 4))
```



0	1
1	2
2	3
3	4

dtype: int64

When "data" is Dictionary.

The keys become the index of the series, and the values become the data.

Code Snippet:

```
pd.Series({'a': 1, 'b': 2, 'c': 3})
```



series:

a	1
b	2
c	3

dtype: int64

When "data" is a scalar Value.

If a scalar value is passed, the value is repeated to match the length of the index if an index is provided.

Code Snippet:

```
pd.Series(10, index=['a', 'b', 'c'])
```



series:

a 10

b 10

c 10

dtype: int64

The **index** parameter

The index parameter accepts

an array-like structure.

This includes any one-dimensional

sequence or iterable, such as

a list, tuple, or numpy array.

```
data = [1, 2, 3]
```

```
index = ['a', 'b', 'c']
```

```
pd.Series(data, index=index)
```



series:

a 1

b 2

c 3

dtype: int64

The index parameter can also be a pandas Index object. This is a specific type of object that pandas uses to manage the labels of Series and DataFrames.

```
data = [4, 5, 6]
index = pd.Index(['x', 'y', 'z'])
pd.Series(data, index=index)
```

series:

x	4
y	5
z	6

dtype: int64

Note that the Index is first build with the keys from the dictionary. After this the Series is reindexed with the given Index values, hence we get all NaN as a result.

```
data = {'a': 1, 'b': 2, 'c': 3}
index=['x', 'y', 'z']
pd.Series(data=data, index=index)
```

series:

x	NaN
y	NaN
z	NaN

dtype: float64

The **dtype** parameter

The dtype parameter, which can be a string, numpy.dtype, or ExtensionDtype, is optional and defines the data type for the resulting

If not specified, this will be inferred from data.

*#The dtype parameter is set to "float64", so the Series is created with
#floating-point numbers, even though the input data consists of integers.*

```
pd.Series([1, 2, 3], dtype=float)
```

```
0    1.0
1    2.0
2    3.0
dtype: float64
```

#If not specified, this will be inferred from data.

```
pd.Series([1, 2, 3])
```

```
0    1
1    2
2    3
dtype: int64
```

The **name** parameter

The name parameter, which is hashable and defaults to None, specifies the name to be assigned to the Series.

The name parameter must be hashable, meaning it can be a string, number, or tuple, but not a list or dictionary.

series without name::

```
pd.Series([10, 20, 30])
```



```
0    10
1    20
2    30
dtype: int64
```

series with name:

```
pd.Series([10, 20, 30], name="MySeries")
```



```
0    10
1    20
2    30
Name: MySeries, dtype: int64
```



```
#Accessing the Name: You can access the name of the Series using "series.name"  
series1=pd.Series([10, 20, 30], name="MySeries")  
series1.name
```

```
'MySeries'
```

```
#without names  
series1=pd.Series([10, 20, 30])  
series1.name
```

The copy parameter

**The "copy" paramete is boolean and defaults to False.
It is used to copy input data.It Only affects Series or 1d ndarray input.**

When copy=False (Default Behavior)

Constructing Series from a list with copy=False.
Due to input data type the Series has a copy of the original data even though copy=False, so the data is unchanged.

```
data1 = [1, 2, 3]  
ser = pd.Series(data1, copy=False)  
ser
```



0	1
1	2

Constructing Series from a 1d ndarray with copy=False.
Due to input data type the Series has a view on the original data, so the data is changed as well.

```
data2= np.array([1, 2, 3])  
series2 = pd.Series(data=data2, copy=False)  
series2
```



0	1
1	2

```

2    3
dtype: int64

ser.iloc[0] = 999
ser

```



```

0    999
1     2
2     3
dtype: int64

```

data1



```
[1, 2, 3]
```

```

2    3
dtype: int32

series2.iloc[0] = 999
series2

```



```

0    999
1     2
2     3
dtype: int32

```

data2



```
array([999,  2,  3])
```

When 'copy=True'

When copy=True, a new copy of the input data is made. Modifying the Series will not affect the original input data.

```
data1 = [1, 2, 3]
ser = pd.Series(data1, copy=True)
ser
```



```
0    1
1    2
2    3
dtype: int64
```

```
ser.iloc[0] = 999
ser
```



```
0    999
1     2
2     3
dtype: int64
```

```
data1
```



```
[1, 2, 3]
```

```
data2= np.array([1, 2, 3])
series2 = pd.Series(data=data2, copy=True)
series2
```



```
0    1
1    2
2    3
dtype: int32
```

```
series2.iloc[0] = 999
series2
```



```
0    999
1     2
2     3
dtype: int32
```

```
data2
```



```
array([1, 2, 3])
```

The `fastpath` parameter

s a user, you typically don't need to worry about or directly interact with the `fastpath=_NoDefault.no_default` parameter. It's a part of pandas' internal mechanisms to ensure that operations are efficient and that object creation is optimized under the hood. If you're exploring the source code or debugging deep into pandas' internals, you might encounter this, but for regular usage, it's abstracted away.

```
#_NoDefault.no_default: This is a sentinel value used by pandas to indicate that  
# no specific value was provided for the fastpath parameter. The _NoDefault class is a  
# way to define default behavior without explicitly using common defaults like None.
```

Sources & References

[pandas.Series Documentation](https://pandas.pydata.org/docs/reference/api/pandas.Series.html) (<https://pandas.pydata.org/docs/reference/api/pandas.Series.html>)

Author Biography

Kichere Magubu is a data enthusiast and content creator.

Other E-books

[Download](https://github.com/kicherethedatascientist/My_e-Books) (https://github.com/kicherethedatascientist/My_e-Books)

Contacts and Social Media

Kichere Magubu

 Dar es salaam, Tanzania

 [Kichere Magubu](#)

 [Kichere The Data Scientist](#)

 [KichereTheDataScientist](#)

 [KichereTheDataScientist](#)

 kicherethedatascientist@gmail.com

 [kicherethedatascientist](#)

 +255 654 729 851

Powered by

[Eastern Africa Statistical Training Centre](#)

[SKT Tanzania Ltd](#)

Thank you!

Thank you for reading this e-book!

If you found this book valuable and beneficial, I kindly request that you take a moment to review it. Your honest feedback is

highly appreciated and makes a significant difference to me.

Please take a minute to write me an honest review. Your support means the world to me!

Thank you!

In []: